

Doubling a 6502 Clock to 28 MHz with Dual-Phase CPU Operation

Conceptual hardware design for two 14 MHz 6502-class processors operating on opposite phases of a 28 MHz clock, with shared memory implemented using dual-port RAM or equivalent banked/arbitrated memory.

Important: This is an architectural design study, not a claim that an original NMOS 6502 can safely run at 14 or 28 MHz. The practical implementation requires a modern high-speed 6502-compatible core or a suitably fast CMOS/FPGA implementation, and the exact bus timing must be verified against the chosen part's datasheet.

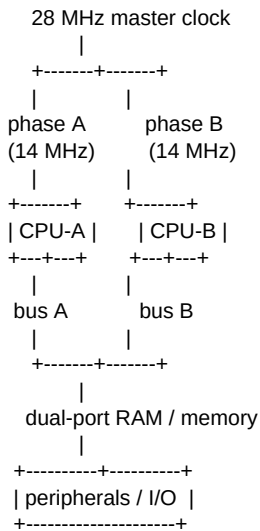
1. Core idea

Instead of trying to make one 6502 execute at 28 MHz, use two CPU instances. CPU-A receives the rising-phase clock and CPU-B receives the falling-phase clock. Each CPU therefore runs at 14 MHz, while the system can schedule one CPU transaction during each half-cycle of the 28 MHz source clock. This is analogous to interleaving two workers on opposite clock phases.

Clock relationship

28 MHz source	↑	↓	↑	↓	↑	↓
CPU-A	active	hold	active	hold	active	hold
CPU-B	hold	active	hold	active	hold	active

2. Recommended architecture



3. Why dual-port RAM helps

A true dual-port RAM gives CPU-A and CPU-B independent address/data/control interfaces. That avoids putting both processors directly onto one electrical bus and eliminates a large amount of bus arbitration logic. The RAM must still define behavior when both ports access the same location at nearly the same time. For ordinary RAM this is usually handled by a documented collision mode, arbitration, or by software/hardware rules that prevent simultaneous writes.

Suggested memory map

Region	Purpose	Access
\$0000-\$7FFF	Shared RAM / program data	Both CPUs
\$8000-\$BFFF	ROM or shared code	Both CPUs

Region	Purpose	Access
\$C000-\$C0FF	CPU-A mailbox / control	Both, controlled
\$C100-\$C1FF	CPU-B mailbox / control	Both, controlled
\$C200-\$C2FF	System timers / interrupt control	Both
\$C300-\$FFFF	Peripherals / expansion	Arbitrated

4. Clock generation and phase control

Use a clean 28 MHz oscillator or PLL-generated clock as the master timing reference. Generate complementary 14 MHz phase clocks with a flip-flop or dedicated clocking resource rather than relying on arbitrary combinational inversion. The two CPU clock paths should have controlled skew and, where required by the chosen CPU, non-overlap. Avoid routing a generic logic-gate-inverted clock through long PCB traces as the primary clocking method.

Key point: The phrase 'double the clockspeed' is slightly misleading here. Each 6502 runs at 14 MHz; the aggregate interleaved memory-access opportunity is 28 MHz. You do not get the performance of a single 28 MHz 6502 automatically, because each CPU has its own architectural state and software must exploit both processors.

5. Bus and memory timing

The central timing problem is not merely generating two clocks. A 6502 bus cycle has address-valid, read/write, data-valid, and setup/hold relationships. The memory and glue logic must meet those requirements at the selected CPU clock rate. At 14 MHz, a full CPU cycle is approximately 71.43 ns. The half-period of the 28 MHz master is approximately 17.86 ns, so the phase-to-phase handoff must be designed carefully.

Quantity	Value
28 MHz master period	35.71 ns
28 MHz half-cycle	17.86 ns
14 MHz CPU period	71.43 ns
14 MHz half-cycle	35.71 ns

6. Handling I/O

Do not let both CPUs freely drive the same peripheral bus. A practical scheme is to give each CPU its own bus interface into a central I/O fabric. Reads can be multiplexed; writes should be serialized or directed to separate registers. For devices that have side effects (UARTs, timers, video registers, SD controllers, etc.), define one owner CPU or add explicit arbitration.

7. Inter-CPU communication

Use small mailbox regions in shared RAM. A typical protocol is: CPU-A writes a command and arguments, then sets a flag; CPU-B detects the flag, copies the arguments into local state, performs the operation, writes a result, and clears/updates the flag. If interrupts are available in the selected 6502 implementation, a shared interrupt controller can make this more efficient.

8. Coherency and ownership

Because there is shared RAM, software needs a simple ownership convention. For example, divide the application into private regions and shared message buffers. Avoid having both CPUs continuously modify the same data structure. If a lock is required, use an atomic hardware primitive or a carefully designed mailbox/flag protocol; ordinary 6502 read-modify-write instructions are not automatically a universal multi-master locking mechanism.

9. Reset and synchronization

Both CPUs should share a controlled reset sequence. Release reset synchronously relative to their respective clock phases. If one CPU must start later, give it a separate reset/enable signal. Provide a watchdog or supervisor so that a stalled CPU cannot permanently block shared resources.

10. Practical implementation choices

Approach	Advantages	Trade-offs
Two fast CMOS 6502-compatible CPUs	Simple mental model; real CPU buses	Must verify 14 MHz timing and shared-memory behavior
Two FPGA 6502 cores	Precise clocks, easy arbitration, high speed	Requires FPGA design and careful I/O timing
One FPGA containing two cores + cheap external RAM	Clears interrupt RAM; true dual-port memory	More HDL work; peripheral integration required
One 28 MHz CPU core with time-multiplexed interrupt hardware	Potentially simpler hardware	Not equivalent to two independent CPUs

11. Recommended prototype path

1. Choose the exact 6502-compatible implementation and verify its maximum clock rate, voltage, bus timing, and interrupt behavior.
2. Prototype two CPU cores at 14 MHz with separate memories. Confirm that each core is stable.
3. Replace the separate memories with true dual-port RAM and run independent programs.
4. Add mailbox registers and a shared interrupt mechanism.

5. Add an I/O arbiter/fabric rather than directly tying the CPU buses together.
6. Only after timing is verified, build the PCB with controlled clock distribution and appropriate signal integrity.
7. Measure clock skew, RAM access timing, setup/hold margin, and bus contention with a logic analyzer or oscilloscope.

12. A useful refinement: phase-interleaved memory

If the goal is specifically to make a legacy-style 6502 system perform more work per 28 MHz master cycle, another option is to use phase-interleaved memory access: CPU-A owns one phase and CPU-B owns the other. A memory controller can then present a regular 14 MHz bus to each CPU while internally servicing requests on alternating 28 MHz slots. This is often easier to reason about than physically connecting two CPU buses together.

13. Bottom line

The robust architecture is: **28 MHz master clock → two complementary 14 MHz CPU clocks → two independent 6502-compatible CPU buses → true dual-port RAM/shared-memory controller → controlled I/O fabric**. The design gives two processors an interleaved opportunity to access memory every 17.86 ns, but it should be described as a dual-CPU 14 MHz system rather than a single 28 MHz 6502.

Next engineering step: select the exact CPU/core and dual-port RAM part numbers. The timing equations and schematic can then be turned into a concrete logic diagram with address/data/control pins, phase generation, arbitration, reset, interrupts, and memory-cycle timing.